Efficient Range Deletes in LSM Engines

Yu-Cheng Huang
Boston University
USA

****
Boston University
USA

****
Boston University
USA

ABSTRACT

In the realm of modern technology, the LSM-Tree (Log-Structured Merge-Tree) is a notable structure for its exceptional data insertion capabilities, widely utilized in numerous database systems. However, the LSM-Tree faces challenges with range deletions. These deletions are stored in Sorted String Table (SST) files on the disk and brought into the memory only when the file is opened. If the file is closed and reopened again, these range deletes are repetitively read which can lead to extra disk Input/Output (IO) operations, slowing down the system's performance.

Addressing this inefficiency, this paper presents the Range Delete Filter (RDF), a solution designed to cut down disk IOs in the process of finding deleted keys. The RDF serves as an advanced mechanism that quickly identifies and handles deleted keys, thus improving the LSM-Tree's performance. Our objective with integrating RDF is to enhance the point-query efficiency of the LSM-Tree, especially in situations involving frequent range deletions, which are prevalent in dynamic database environments. This paper delves into the architecture and implementation of RDFs, its incorporation into the LSM-Tree, and assesses its impact on the overall system performance through comprehensive experimental evaluations.

ACM Reference Format:
Yu-Cheng Huang, ****, and ****. 2024. Efficient Range Deletes in LSM Engines. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

LSM-based Key-Value Stores. The evolution of technologies like 5G, AGI, and XR has led to an exponential increase in data generation and storage requirements. Many database systems have adopted Log-Structured Merge-tree (LSM) due to its efficiency in write operations. In LSM, data is initially written to a buffer; once the buffer is full, it is flushed to disk as an immutable Sorted String Table (SSTable) with sorted keys. LSM trees feature several levels, with each level being T times larger than the previous one. When a level reaches its capacity, multiple SSTable files are merged and sorted before being moved to the next level.

Range Delete in LSM Trees. Range deletes in LSM trees are handled by storing tombstones that specify the start and end keys, indicating the range of keys to be deleted in overlapping SSTable files. In systems like RocksDB, during compaction, these range delete tombstones can be used to purge obsolete entries across all compacted files.

Problems. A significant challenge in handling range deletes within SSTable files is the potential need to open and read specific files to ascertain the existence of an entry. This process can be slow. If it's possible to establish that a key does not exist in memory before accessing these files, we can expediently return and avoid the time-consuming task of reading files. This disparity in access speeds between disk and memory is a crucial factor that can decelerate query response times. By identifying non-existent keys earlier, unnecessary disk reads can be circumvented, thereby enhancing the overall efficiency of the system.

1.1 Motivation

By storing range delete information in the memory, an RDF offers a streamlined approach for handling queries. It actively filters out queries aimed at keys already marked for deletion, thereby preventing the unnecessary activation of disk-based operations. This approach is particularly beneficial in scenarios where the frequency of range deletions is high, and point queries often intersect with these ranges.

Moreover, this memory-based RDF implementation opens up new possibilities for optimizing the LSM tree's operational dynamics. By allowing for rapid determination of the non-existence of keys, the RDF effectively narrows down the search space for point queries, thus speeding up the overall query response time. This optimization is especially significant in systems with large datasets and frequent read operations, where disk reads constitute a substantial operational bottleneck.

In summary, it not only reduces the number of disk I/O operations during point queries but also enhances the efficiency of the entire system by enabling quicker and more effective data searching processes.

1.2 Problem Statement

The search for a key, given the presence of range delete data in SSTable files, may be inefficient on LSM storage. Point queries initiate their searching in memory and then progress to disk if necessary. The organizational structure of these files spans multiple levels, and point queries systematically move from the lower to higher levels in this hierarchy. When a query is made, a relevant SSTable file on each level is targeted when

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

its range, spanning from its minimum key to its maximum key, encompasses the queried key. The process involves reading range delete information from the file's metadata to ascertain existence of the key. However, this approach can lead to inefficiencies, particularly when there's repetitive reading of range delete data or when multiple files across different levels are opened to retrieve this information and search for the key, resulting in unnecessary I/O operations.

A more efficient approach would be to store range delete information directly in memory. This method could sidestep the need for opening certain files , thereby reducing I/O operations. Furthermore, considering range delete and point query operations coming in order of their issued sequence, associating sequence numbers with range delete tuples, the pair of start and end keys, becomes redundant because data on the lower level is always newer than the ones on the higher level. Eliminating the use of sequence numbers in Range Delete Filters (RDFs) allows for the merging of overlapping ranges within the same level, thereby further enhancing system efficiency and reducing the operational overhead.

1.3 Contributions

This study aims to: (a) Discuss and analyze the impact of different RDF implementations (PLRDF, split-PLRDF, toplevel-RDF, Skyline-RDF) on system performance and their memory footprints. (b) Eliminate the sequence number associated with each range to enable range merging in RDFs to decrease its memory footprint.

2 LSM BACKGROUND

The LSM tree is a structure optimized for writes and consists primarily of two operations: Flush and Compaction. It consists of multiple levels with T-ratio up-growing capacity.

Leveling and Segmented LSM. Data is stored in SSTable files on each level, with non-overlapping ranges between files. During compaction, only selected files are moved to the next level.

Flush. When the buffer is full, data is sorted and written into an immutable sorted string table on disk.

Compaction. Triggered when a level is full, several files will be picked out following some rules and sort-merged with the overlapping files on the next level.

Write Amplification and Space Amplification. Space amplification occurs due to the presence of the same key on different levels. Write amplification happens when duplicate keys in the database result in additional writes when moving data to the next level.

Range Delete. Range deletes are stored as range tombstones, represented by a triple of start value, end value, and sequence number in each SSTable file that contains their ranges.

3 PROBLEM

In rocksDB, range deletes hold information about non-existent entries, but they are solely stored in files, resulting in additional disk IO operations in accessing them, inevitably declining query speed. Presently, the bloom filter only stores details about point entries, and fencing pointers store information about existing file ranges. However, there is no filter that stores delete ranges, making it impossible to determine whether a specific point is deleted without resorting to disk IO.

First, current system loss a degree of freedom for not keeping range delete information in the buffer. One advantage of rocksDB is that users can choose to keep the bloom filter inside the buffer, enabling them to filter out non-existent keys in a given file without needing direct disk access. Unfortunately, this option is not available for range deletes. When a user wants to determine if a key is deleted by a range delete or not, performing disk IO operations becomes necessary. Consequently, we lose the freedom to remove range-deleted keys before directly accessing the disk, leading to potential impacts on performance.

In Addition, without range delte filter, systems take more time in reading an SST file. For a system, files are read in blocks and each file contains multiple blocks. To read an SST file, rocksDB first reads the metadata to identify which block may contain the target key or have overlapping deleted ranges with it. Then, it proceeds to find the key in the block that may contain the key and retrieve range delete information in range delete block. Since range deletes are currently stored with files on each level, the system may perform extra disk IOs to pinpointt the file where either the searching key or range deltes in interest lie in to determine whether a key exists or deleted.

4 SOLUTION NAME

4.1 Solution Design

The flow of the point query in RocksDB, as illustrated in 1, involves several steps. Initially, the key is searched in the memtable, followed by a search in the SST from level 0 to the bottom level. The file range metadata on each level is consulted to identify a potential wfile containing the key. If the file is not present in the buffer, it is opened with its index block, RD, and bloom filter prefetched into the buffer. The RD information forms the RDF that used to filtered out the deleted keys. The key undergoes filtering through the bloom filter and index block before initiating a disk IO to retrieve the data block from the SST. Subsequently, the existence of the data is checked with RDF, and if found, the process concludes; otherwise, it proceeds to the next level.

For PLRDF, Split-PLRDF, and TopLevel-RDF, RD is stored on levels without a sequence number. This design allows overlapping RD to be consolidated, reducing the memory footprint in the buffer. However, without a timetag, these RDFs only support PQ that occurs after all previous insertions and RD has been written into the database in order.

In the case of PLRDF, Split-PLRDF, RDs are stored on each level, while TopLevel-RDF stores RD only on the top

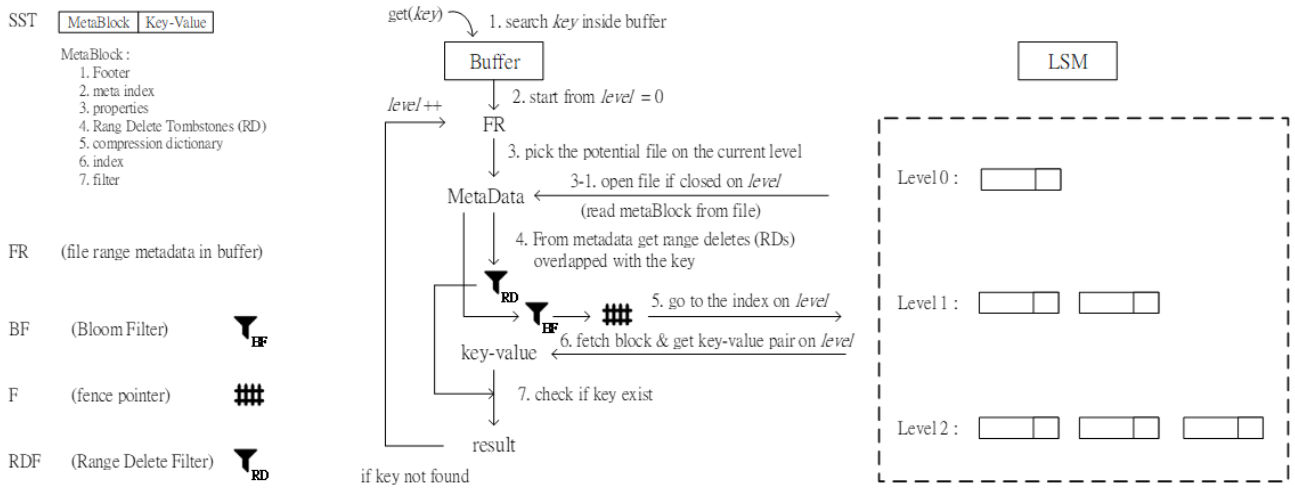


Figure 1: The flow of the PQ in RocksDB.

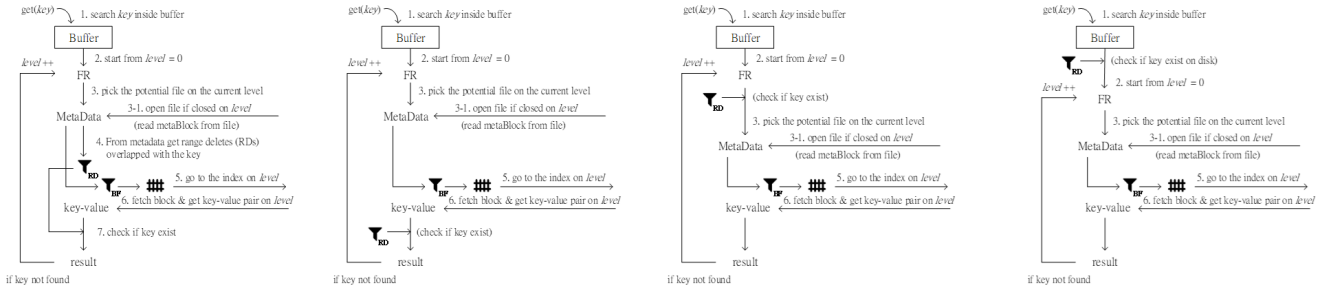


Figure 2: The difference of the PQ flow for each of the RDF.

level. During range deletes flushed to disk, range deletes are merged and stored on level 0 in PLRDF. Subsequently, during compaction, corresponding range deletes are moved and merged to the next level vector. During flush or compaction, range deletes from the top the k level remove obsolete entries from the files they compacted to on the k+1 level, so that no keys that can be deleted by the RD in the same file exist.

Figure 3 illustrates the PQ flow for each RDF, highlighting differences in how they check if a key is deleted. These differences are outlined below:

RocksDB(None): Before searching for a key on a given level, range deletes covering the key are collected from the SST, incurring a disk IO if the data is not in the buffer. These range deletes, along with their timestamps, are then compared with the retrieved data to determine if the key is deleted.

PLRDF: Similar to RocksDB, but range deletes are stored in the buffer, eliminating the need for disk IO to retrieve RD information. RDF is checked after each retrieval from the SST to validate that the retrieved data is not deleted.

Split-PLRDF: Due to the splitting of range deletes when a newer key is added to the SST, RDF can be applied before SST block retrieval, saving disk IOs.

TopLevel-RDF: RDF is stored only on the top level, keeping track of all RD on the disk. The splitting functionality ensures it does not return false negatives for newly inserted keys.

Skyline-RDF: Similar to PLRDF in PQ flow, but RD is stored with a timetag to determine the existence of a key from any level. This RDF is only called when a key is found, without pruning on the searching branches, making it the worst-performing RDF.

4.2 Solution Memory Footprint

Benefiting from the RD filter, there's the cost of memory footprint. In this section, we will discuss the memory footprint of each RDF.

Representing range delete with a tuple or a triple, there's an start key and an end key each takes a byte. For recording

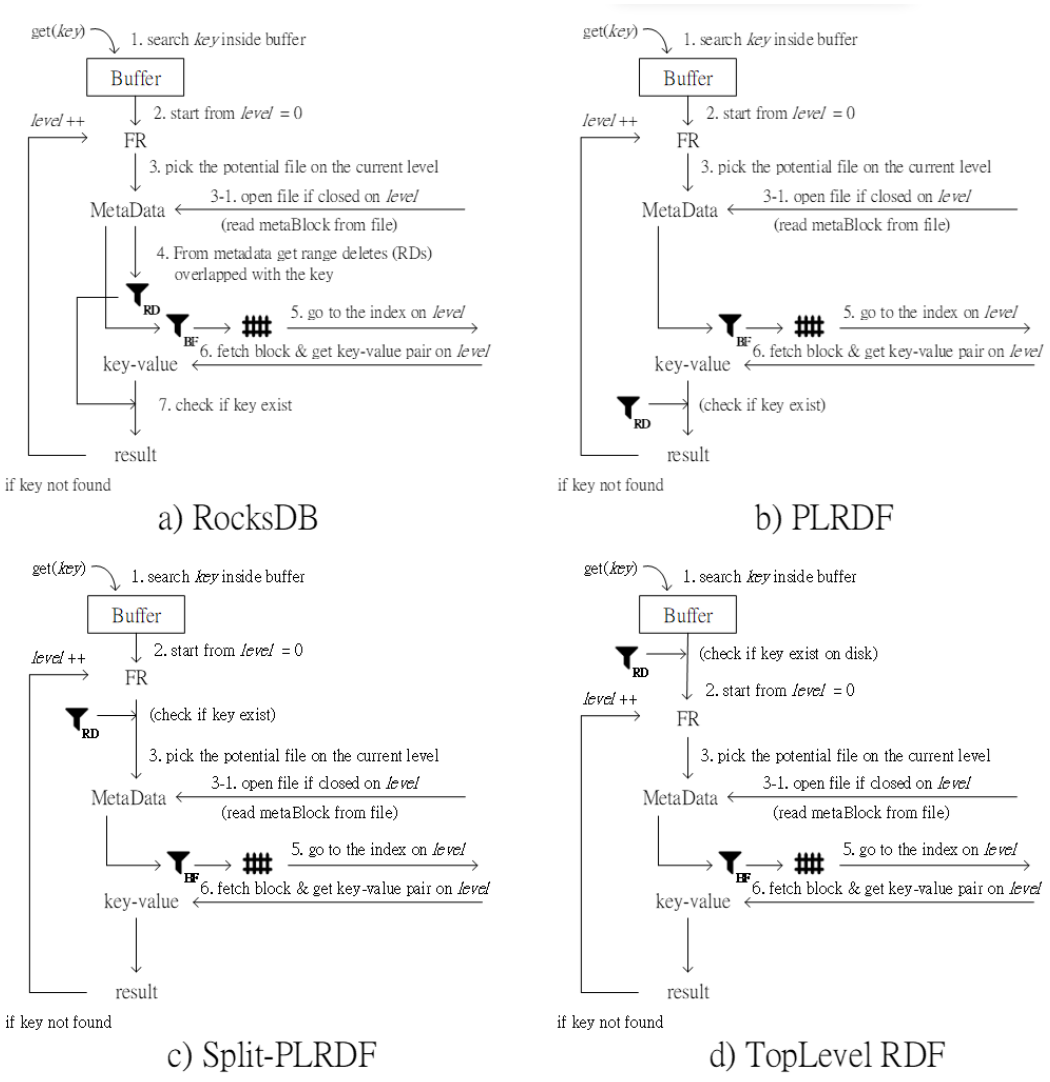


Figure 3: The difference of the PQ flow for each of the RDF.

the order, a sequence number of b byte is required.
Furthermore, let α be the range split ratio in an RDF, β be the range absorb ratio and γ be the inserted-keys-driven split factor. All these ratio may vary with time.
Suppose there are N range deletes in the RDF, then the memory footprint of each RDF is as follows:

	memory footprint
RocksDB(None)	O(#opened files * avg #RD in a file)
PLRDF	$O(N(1-\beta)2\alpha)$
Split-PLRDF	$O(N(1-\beta)(1+\gamma)2\alpha)$
TopLevel-RDF	$O(N(1-\beta)(1+\gamma)2\alpha)$
Skyline-RDF	$O(N(1+\alpha-\beta)(2\alpha-\beta))$

- **RocksDB(None).** Default RocksDB stores range deletes in each SST file where they belong to. Range Deletes are loaded when SST files are opened. The memory footprint depends on the number of opened SST files and the number of range deletes in those files.

- **PLRDF.** Memory footprint = $O(N(1-\beta)2\alpha)$ bytes. Tuples are enough to represent range deletes. When ranges on the same level intersect with one another, they are absorbed into one, which increases the β, range absorb ratio. However, overlapped range deletes may cumulate on different levels, which decreases β.
  It is favorable to have ranges accumulated on limited levels.
- **Split-PLRDF.** Memory footprint = $O(N(1-\beta)(1+\gamma)2\alpha)$ bytes. Similar to PLRDF, but this one suffered from inserted-keys-driven split. When a new key falls in the region of a range delete, it splitted into 2 ranges,

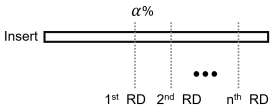


Figure 4: The first RD is inserted at the α of the total insert, and all the following RDs are equally separated between the first RD the end of the insert workload.

which contributes to γ, inserted-keys-driven split factor. It is favorable to have less inserted keys fall into the range deletes.

- **TopLevel-RDF.** Memory footprint = $O(N(1-\beta)(1+\gamma)2\alpha)$ bytes. Like Split-PLRDF, instead ranges don't move down to the next level, so it sufferes more from the inserted-keys-driven split.
  The worst case can be a range keep split by all the inserted keys, causing an amplification of number of inserted keys.
- **Skyline-RDF.** Memory footprint = $O(N(1+\alpha-\beta)(2\alpha-\beta))$ bytes. Triples are used to represent range deletes. When small range falls within the larger range the range split into tree segments which contributes to the α, range split ratio; In opposite, when a larger range comes in and contain the smaller range, the ranges are absorbed into one, which contributes to the β, range absorb ratio.
  It is favorable to have larger ranges come in and absorb the old ranges.

5 EVALUATION

5.1 Experiment Settings

- cpu:
- ram:
- ssd:
- rocksdb:

workload settings:
Workload composed 2 parts, the first part is the insert workload, the second part is the query workload. Within the insertion workload, there're entry insert andRD while the RDs are equally dirstributed with an variable α that decides where the first RD starts at, which is shown in Figure .

In 4.

In the following part, read buffer is disabled. And Point Query is performed after all the insertions has been done.

From Figure 5 and Figure 6, one with SKyline and one without Skyline, we can see that the Skyline RDF is the worst of the all when performing PQ on the existing keys, because it doesn't return when the level contains RD, instead it only return when a key is found or going through all the levels. While the other RDF returns if they find a key is deleted by RD on a certain level. Comparing to Figure 8 which all the RDFs have similar work time, this is due to the fact that when performing PQ on existing keys, there's no chance of

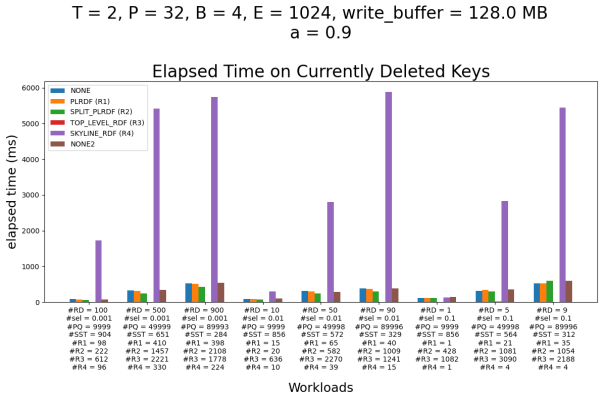


Figure 5: Elapsed time of testing on all of the deleted keys when α = 0.9, T = 2, P =32, E = 1024. From left to right separated into 3 groups [(g1,g2,g3), (g4,g5,g6), (g7,g8,g9)], each of them has selectivity 0.001, 0.01, 0.1 respectively. While (g1,g4,g7) has 10% of total keys deleted and the middle three (g2,g5,g8) have 50%, whereas the tallest ones (g3,g6,g9) have 90% keys deleted.

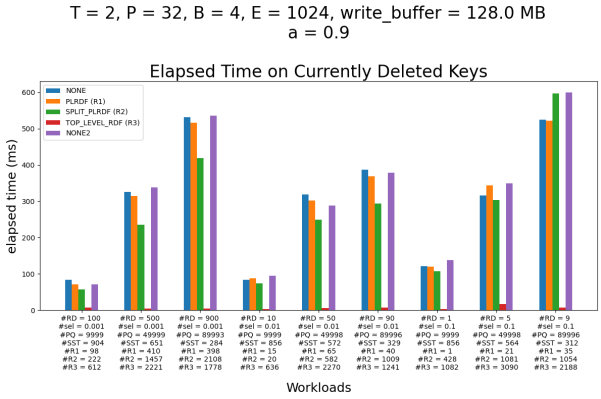


Figure 6: Elapsed time of testing on all of the deleted keys without Skyline when α = 0.9, T = 2, P =32, E = 1024. From left to right separated into 3 groups [(g1,g2,g3), (g4,g5,g6), (g7,g8,g9)], each of them has selectivity 0.001, 0.01, 0.1 respectively. While (g1,g4,g7) has 10% of total keys deleted and the middle three (g2,g5,g8) have 50%, whereas the tallest ones (g3,g6,g9) have 90% keys deleted.

early return, which means that all doing the search until they reach the level where key lies.

As the Figure 7 shows, IOs occupies more than half of the time, if we can reduce the number of IOs, we can reduce the total query time.

Diving down to see the numbers of IOs spent on currently deleted keys in Figure 9 and Figure 10, there's no difference

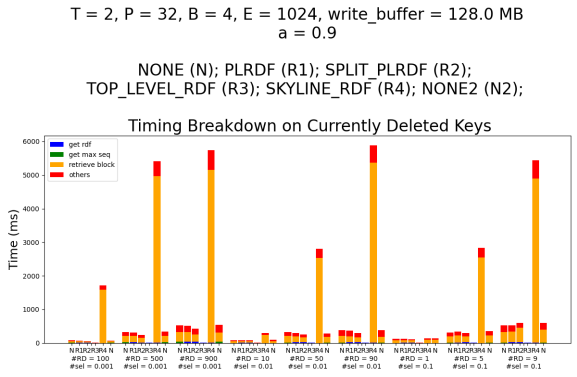


Figure 7: Timing breakdown of testing on all of the deleted keys when α = 0.9, T = 2, P =32, E = 1024. Over 9 workloads.

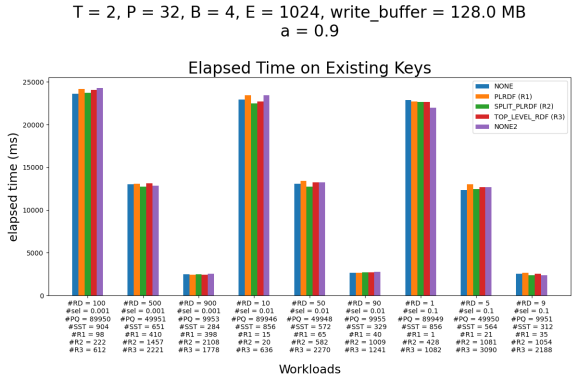


Figure 8: Elapsed time of testing on all of the existing keys when α = 0.9, T = 2, P =32, E = 1024. Over 9 workloads.

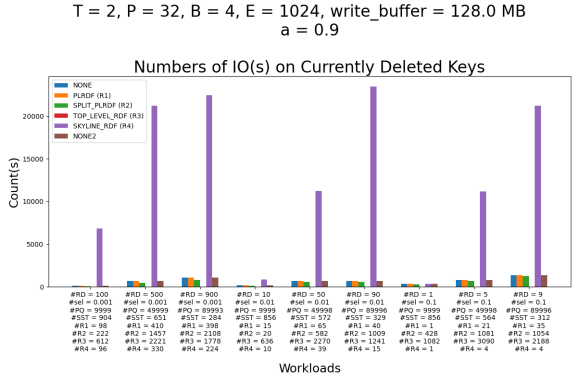


Figure 9: Elapsed time of testing on all of the existing keys when α = 0.9, T = 2, P =32, E = 1024. Over 9 workloads.

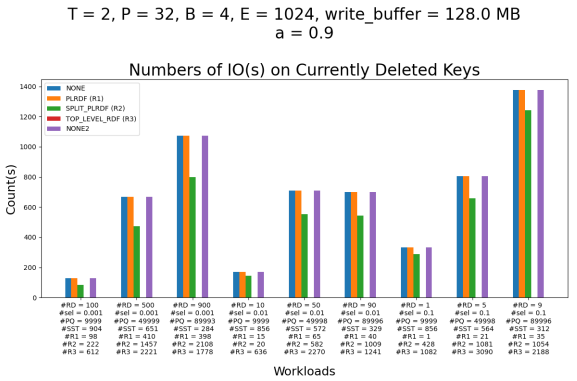


Figure 10: Elapsed time of testing on all of the existing keys when α = 0.9, T = 2, P =32, E = 1024. Over 9 workloads.

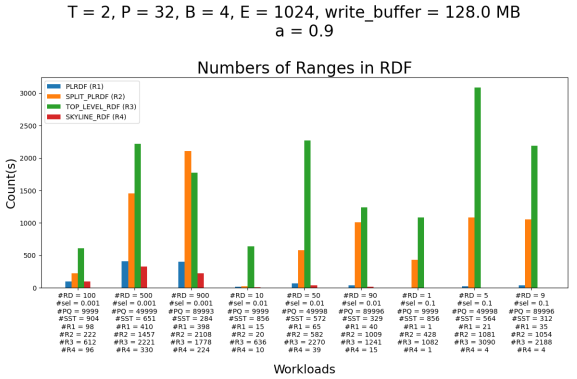


Figure 11: Number of ranges tested on all of the existing keys when α = 0.9, T = 2, P =32, E = 1024. Over 9 workloads.

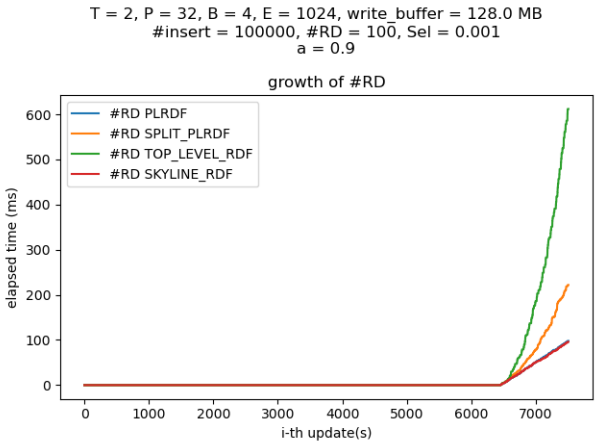


Figure 12: Numbers of ranges in RDs across time tested on all of the existing keys when α = 0.9, T = 2, P =32, E = 1024. Over 9 workloads.

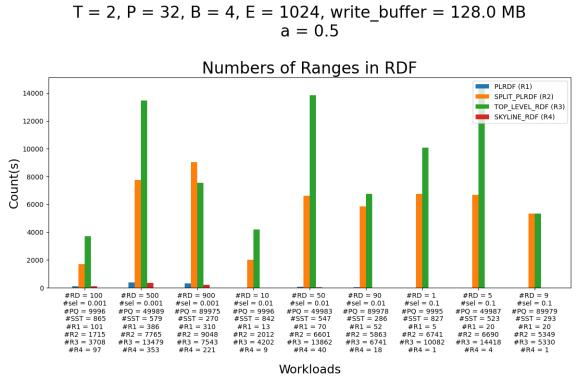


Figure 13: Number of ranges tested on all of the existing keys when $\alpha = 0.5$, T = 2, P =32, E = 1024. Over 9 workloads.

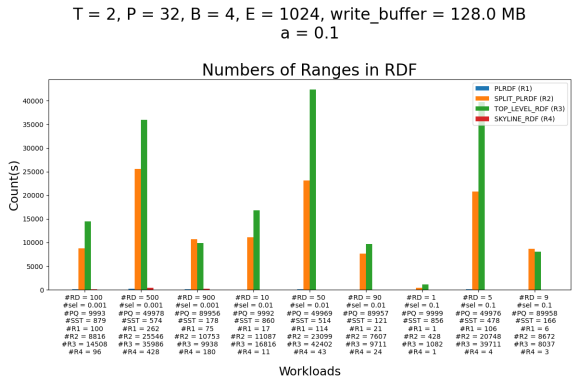


Figure 14: Number of ranges tested on all of the existing keys when $\alpha = 0.1$, T = 2, P =32, E = 1024. Over 9 workloads.

between PLRDF and default RocksDB implementation because when a file is opened the range deletes in a file are all read in to the buffer, when PQ is performed no extra IOs are spent to read in such meta data. Split_PLRDF and TopLevel RDF have less IOs consumption as expected.

When the times goes by, the number of ranges in RDs increases with inserted keys, as Figure 12 shows, the number of ranges in RDs increases almost linearly. The ones split by the inserted points grows faster, and the TopLevel is the worst of all because it stays on the first level and never discard any ranges, while the ranges can be merged if there's a new range overlapped with the old ones.

At the end of the insert, numbers of ranges in RDs are shown in Figure 11. Under most circumstances, TopLevel RDF is the worst for it keeps spliting and never discard any ranges. Skyline seems like the best while the PLRDF is almost comparable with the Skyline. When ranges are merged with a large input RD, numbers of ranges in TopLevel RDF may drops temporarily, but it will soon increase again.

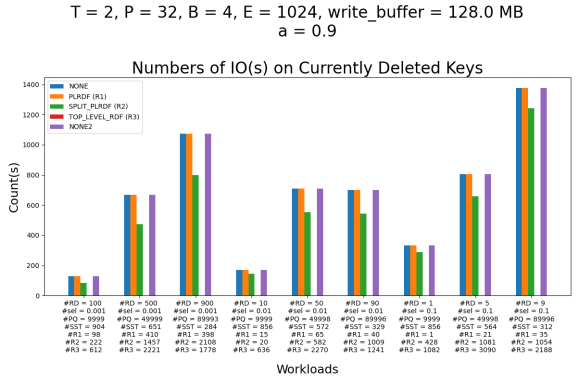


Figure 15: Number of IOs tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, B = 4, E = 1024. Over 9 workloads.

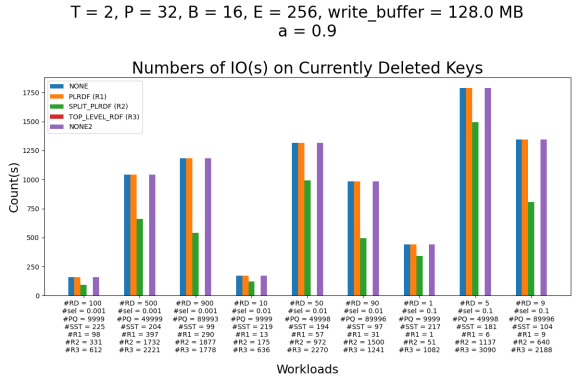


Figure 16: Number of IOs tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, B = 16, E = 256. Over 9 workloads.

Now, when we decrease the value of alpha, making the first RD appeared earlier in the insertion workload, numbers of RDs increases, as in Figure 11, 13 and Figure 14 with α being 0.9, 0.5, 0.1 respectively. The numbers can be 10 times larger.

With Figure 18 and Figure 19, we see when E becomes smaller, the total size of the file becomes smaller, so is the trend of the total elapsed time. While the percentage of CPU times comparing to Disk IOs time decreases too; This may cause by the fact that the retrieved data has to be processed.

6 RELATED WORK

7 CONCLUSION

@onlineRD-rocksDB, author = Abhishek Madan and Andrew Kryczka, title = DeleteRange: A New Native RocksDB Operation, year = 2018, url = https://rocksdb.org/blog/2018/11/21/delete-range.html, urldate = 2018-11-21

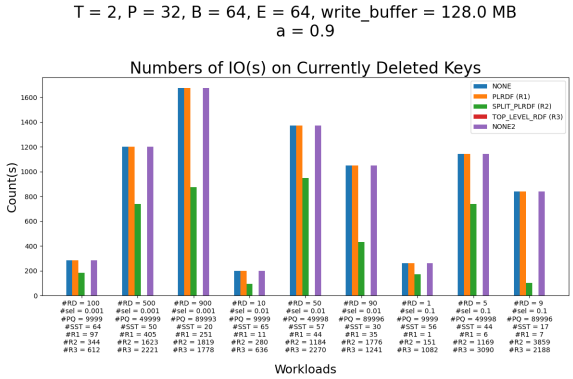
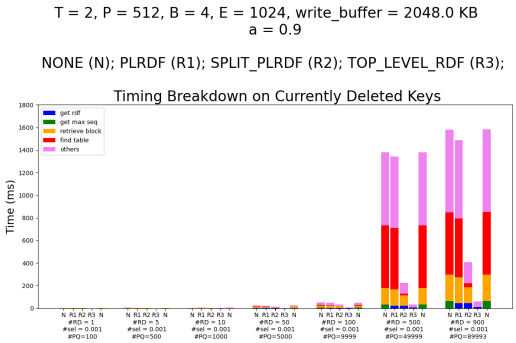


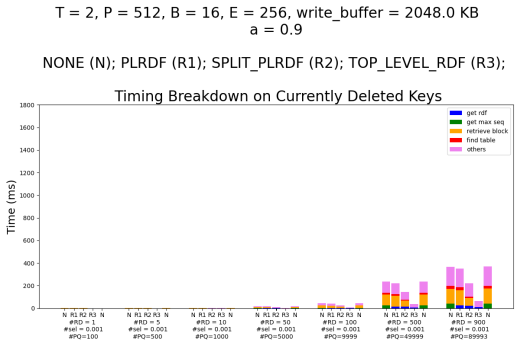
Figure 17: Number of IOs tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 32, B = 64, E = 64. Over 9 workloads.

REFERENCES

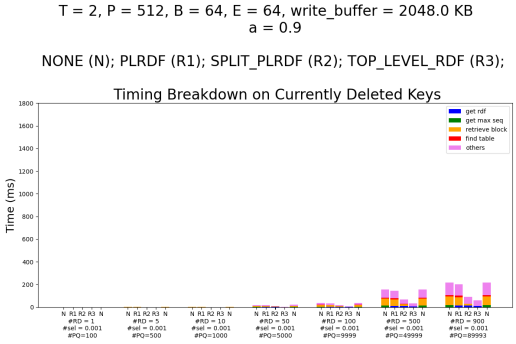
@onlineRD-rocksDB, author = Abhishek Madan and Andrew Kryczka, title = DeleteRange: A New Native RocksDB Operation, year = 2018, url = https://rocksdb.org/blog/2018/11/21/delete-range.html, urldate = 2018-11-21



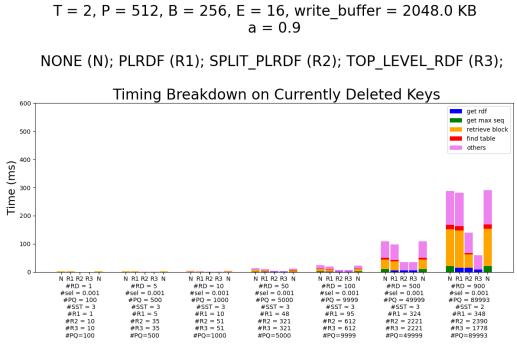
(a) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 4, E = 1024. Over 7 workloads.



(b) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 16, E = 256. Over 7 workloads.

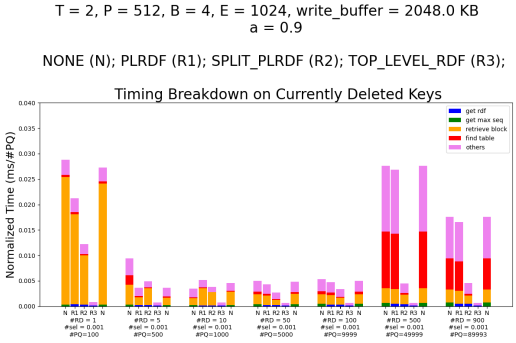


(c) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 64, E = 64. Over 7 workloads.

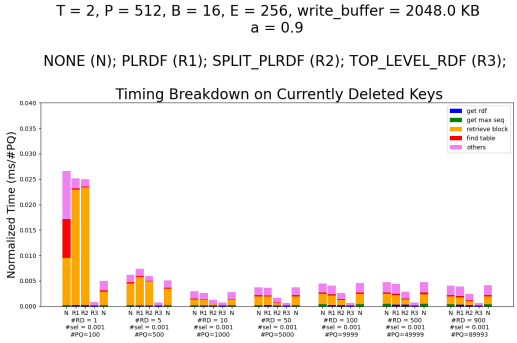


(d) Timing breakdown, tested on all of the existing keys when $\alpha = 0.9$, T = 2, P = 512, B = 256, E = 16. Over 7 workloads.

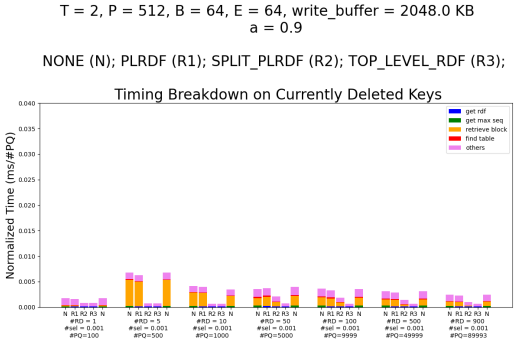
Figure 18: Normalized timing breakdown, varying on numbers of RDs for workload with $\alpha = 0.9$, T = 2, P = 512, BxE = 4096.



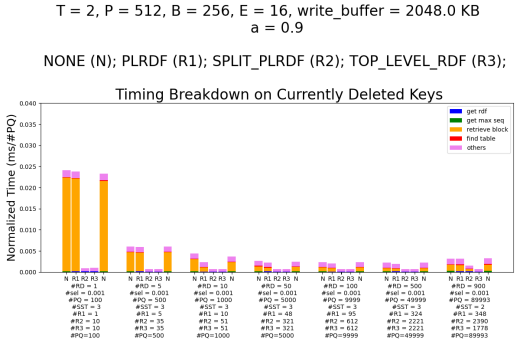
(a) Timing breakdown, tested on all of the existing keys when α = 0.9, T = 2, P = 512, B = 4, E = 1024. Over 7 workloads.



(b) Timing breakdown, tested on all of the existing keys when α = 0.9, T = 2, P = 512, B = 16, E = 256. Over 7 workloads.



(c) Timing breakdown, tested on all of the existing keys when α = 0.9, T = 2, P = 512, B = 64, E = 64. Over 7 workloads.



(d) Timing breakdown, tested on all of the existing keys when α = 0.9, T = 2, P = 512, B = 256, E = 16. Over 7 workloads.

Figure 19: Normalized timing breakdown, varying on numbers of RDs for workload with α = 0.9, T = 2, P = 512, BxE = 4096.